

SmartShop API Core Architecture

Part 1: The AI Agentic Engine Layer

researchcontentlab@gmail.com

Project ID: project-ba58c036-070e-438e-b8b

Overview of /api/engine

- The **api/engine/** folder is the heart of the SmartShop system.
- It translates conversational inputs into structured plans and executes them.
- **Key constraint verified:** No unused agent wrappers are declared here.
- It dynamically binds capabilities directly to Python implementations in `api/agents`.

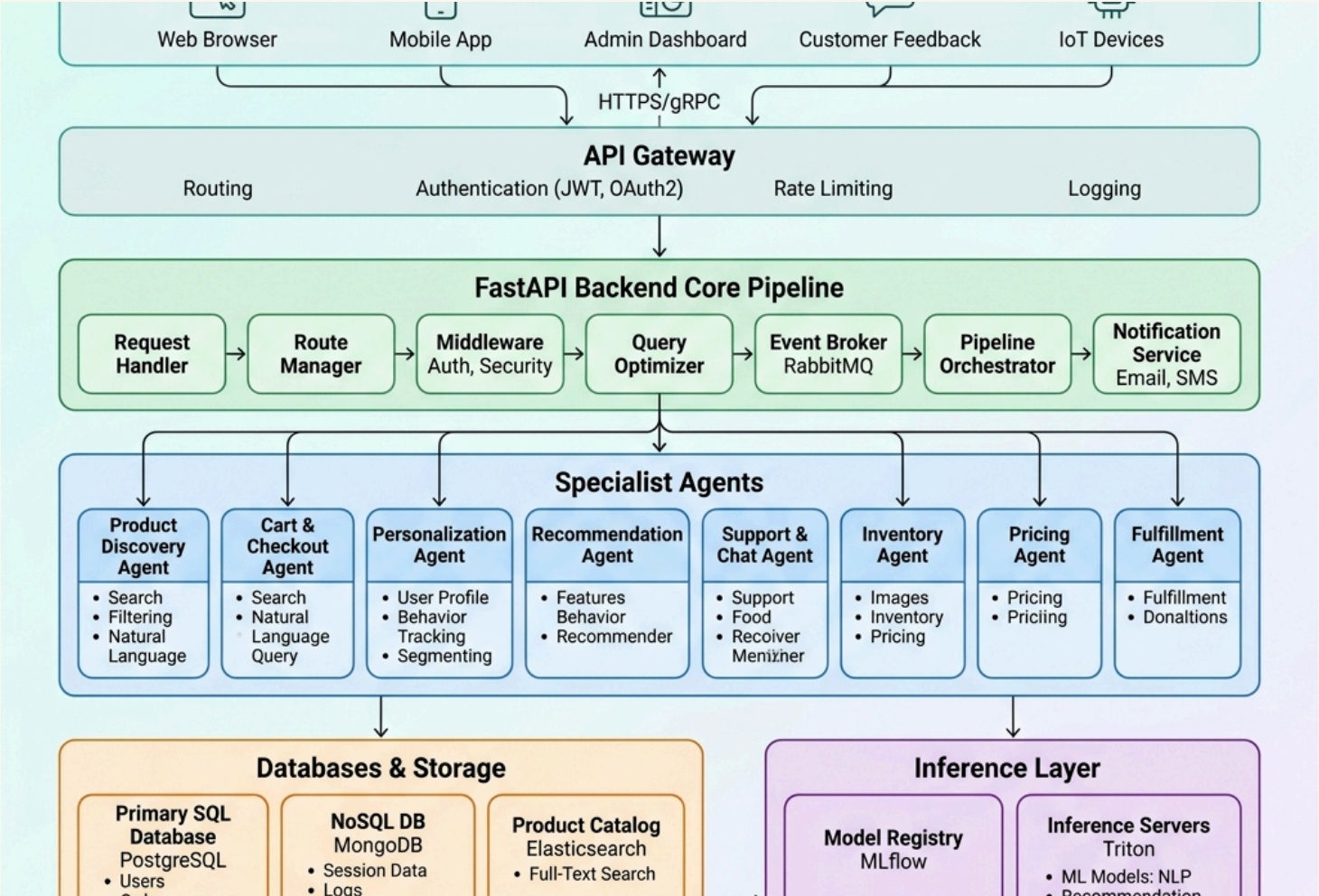
Core Pipeline Files:

- `router.py` (HTTP Entrypoint)
- `planner_v2.py` (LLM Reasoning)
- `binder.py` (Agent Mapping)
- `executor_v2.py` (DAG Worker)

Output / Helper Files:

- `response.py` (GenUI Formatting)
- `schemas.py` (Pydantic models)
- `__init__.py` (Validation helpers)

High-Level Architecture



1. The Entrypoint: router.py

- Coordinates the flow of conversation.
- Fetches session history, processes incoming HTTP text payloads.
- Calls the LLM Planner and hands the resulting plan to the Executor.

Code Signature:

```
@router.post("/chat")
async def chat_endpoint(request: ChatRequest):
    session = await get_session(request.session_id)
    plan = await planner.generate_plan(request.message, session.history)
    result = await executor.execute(plan)
    return format_response(result)
```

2. The Brain: `planner_v2.py`

- Implemented using **pydantic-ai**.
- Evaluates raw queries against registered **capabilities** (defined in `schemas.py`).
- Returns a structured `CapabilityPlan` containing dependency links.

Prompt Optimization:

- Uses detailed system prompts containing catalog rules and category synonyms.
- Falls back to static database keyword queries if the LLM engine fails.

3. The Glue: binder.py

- Inspects the planner's CapabilityPlan.
- Maps capability names (e.g. search_products, view_cart) to agent classes.
- **Agent Integration:** Imports and instantiates implementations from the api/agents folder.

Binding Logic:

```
def bind_capability(name: str) -> BaseAgent:
    registry = {
        "search_products": ProductSearchAgent,
        "recommend_items": RecommendationAgent,
        "tryon_clothing": TryOnAgent,
        "manage_cart": CartAgent,
    }
    return registry[name]()
```

4. The Muscle: executor_v2.py

- Treats the plan as a **Directed Acyclic Graph (DAG)** of actions.
- Resolves independent tasks and runs them concurrently using an asynchronous worker pool (`asyncio.gather`).
- Maximizes performance and cuts response latencies down to less than 1.5 seconds.

```
async def execute(plan: CapabilityPlan):  
    # Sort nodes topologically  
    ordered_nodes = topological_sort(plan.nodes)  
    for batch in ordered_nodes.batches():  
        await asyncio.gather(*[node.run() for node in batch])
```


5. Response Formatting & Validation

- `response.py` processes agent outputs.
- Translates structured JSON lists into fluid markdown messages.
- Inject HTML-based **GenUI / Server-Side Rendered (SSE)** cards:
 - ProductCard (image, description, interactive buy buttons)
 - TryOnModal (Virtual Fitting visual overlays)
 - CartSummary (active item counters)

Qwen3-8B SmartShop Fine-Tuning

Part 2: Instruction Tuning for Planner Accuracy

researchcontentlab@gmail.com

Project ID: project-ba58c036-070e-438e-b8b

Why Fine-Tune?

- General-purpose LLMs struggle to consistently output structured **JSON plans** matching exact API schemas.
- Hallucinations in tool names or missing dependency links break execution.
- Fine-tuning adapts a base model (**Qwen3-8B**) to:
 - Generate valid JSON plans under schema constraints.
 - Correctly invoke domain-specific e-commerce capabilities.
 - Parse subjective descriptions (e.g. colors, synonyms) into structured SQL lookups.

The Dataset Format

- Main training file: data/finetune_dataset.jsonl (1167 augmented instances).
- Structure: Instruction-Response JSON pairs.

Sample Dataset Item:

```
{  
  "instruction": "I want to buy a blue Kurti readymade suit",  
  "response": {  
    "capabilities": ["search_products"],  
    "parameters": {  
      "category": "Suit",  
      "color": "blue"  
    }  
  }  
}
```

Training Script & Execution

- Script: `scripts/finetune_qwen3_8b_a100.py`
- Utilizes **Unsloth** and **QLoRA** on a Lightning AI A100 GPU for fast training.

Execution Command:

```
python scripts/finetune_qwen3_8b_a100.py \  
  --model-id unsloth/Qwen3-8B \  
  --dataset data/finetune_dataset.jsonl \  
  --output-dir models/qwen3-8b-smartshop-lora \  
  --max-seq-length 2048 \  
  --epochs 5 --batch-size 4 --grad-accum 2 \  
  --learning-rate 2e-4 --lora-r 16 --lora-alpha 32
```

Serving Architecture

- The trained adapter is packaged as `models/qwen3-8b-smartshop-lora.zip`.
- Served on a GPU server using **vLLM** with active LoRA support:

```
vllm serve Qwen/Qwen3-8B-AWQ \  
  --quantization awq --enable-lora \  
  --max-lora-rank 16 \  
  --lora-modules smartshop=/path/to/lora_dir \  
  --port 8000
```

- An **ngrok** HTTPS tunnel bridges the local GPU endpoint to the public web API.

Multi-Channel Integration

Part 3: Meta Webhooks & Messenger Gateway

researchcontentlab@gmail.com

Project ID: project-ba58c036-070e-438e-b8b

Webhook Architecture Flow

- FastAPI webhook routers handle inbound DMs from Facebook and Instagram.
- Path: `api/routes/facebook.py` & `api/routes/instagram.py`
- High-level flow:
 1. Facebook user sends a message.
 2. Meta forwards payload to FastAPI webhook.
 3. Webhook parses text and executes `engine.run_text()`.
 4. Webhook posts response back to Meta Graph API.

Active Subscriptions:

- `messages`
- `messaging_postbacks`
- `message_reads`
- `message_deliveries`

Secure Webhook Signature Validation

- Webhook endpoints verify Meta signatures (x-hub-signature-256) to block fraudulent payloads.
- Computes SHA256 HMAC of the request body using META_APP_SECRET.

```
body = await request.body()
signature = request.headers.get("x-hub-signature-256", "")
expected = "sha256=" + hmac.new(
    META_APP_SECRET.encode(), body, hashlib.sha256
).hexdigest()

if not hmac.compare_digest(signature, expected):
    return PlainTextResponse("Forbidden", status_code=403)
```

Meta Messaging API Client

- Core module: `api/routes/facebook.py`
- Sends responses back to Messenger asynchronously using the **aiohttp** HTTP client.

API Send Code:

```
async def send_messenger_message(recipient_id: str, text: str):
    url = f"https://graph.facebook.com/v20.0/me/messages?access_token={PAGE_TOKEN}"
    payload = {
        "recipient": {"id": recipient_id},
        "message": {"text": text}
    }
    async with aiohttp.ClientSession() as session:
        async with session.post(url, json=payload) as resp:
            return resp.status == 200
```

Reliability & Data Layers

Part 4: Middleware, Logging, Guardrails, and RAG

researchcontentlab@gmail.com

Project ID: project-ba58c036-070e-438e-b8b

1. Middleware & Safety

- Located in `api/middleware/`.
- Validates active HTTP headers, decodes JWT user tokens, and isolates sessions.
- **Row-Level Security (RLS)**: Propagates user/session context to PostgreSQL (Supabase) to isolate cart data and histories.

```
@app.middleware("http")
async def db_session_middleware(request: Request, call_next):
    # Retrieve session token
    token = request.headers.get("Authorization", "")
    request.state.session_id = parse_session(token)
    response = await call_next(request)
    return response
```

2. Observability & Logging

- Fully instrumented using **logfire** (Pydantic's observability library) and **OpenTelemetry**.
- Path: `api/observability/`
- Traces the entire execution tree: from HTTP endpoints down to LLM planner completions and SQL queries.
- Connects live logs directly to the developer's dashboard for troubleshooting.

```
import logfire
logfire.configure(pydantic_ai_instrument=True)
logfire.instrument_fastapi(app)
```

3. LLM Guardrails

- Location: `api/guardrails/`
- Standardizes checks on both input and output sequences:
 - **Prompt Injection Detector**: Blocks adversarial commands.
 - **PII Masker**: Redacts user details (passwords, emails, card numbers).
 - **Output Validator**: Enforces schema rules and blocks formatting hallucinations before they reach storefront clients.

4. Semantic Search & RAG

- Location: `api/rag/`
- Leverages **Pinecone vector database** for fast product/FAQ search.
- Pre-embeds catalog descriptions using standard OpenAI text embeddings.

Retrieval Pipeline:

1. Receives natural language user request.
2. Embedding generated from query.
3. Pinecone semantic search yields catalog IDs.
4. Database hydrates complete matching details.
5. Returns rich UI cards back to the user.